functions, we can configure the destination name in RabbitMQ. Open the application.yml file and configure the binding as follows.

**Listing 10.24 Configuring Cloud Stream output binding**

```
spring:
  cloud:
    function:
      definition: dispatchOrder
    stream:
      bindings:
        dispatchOrder-in-0:
          destination: order-dispatched
          group: ${spring.application.name}
        acceptOrder-out-0:                    ❶
          destination: order-accepted         ❷
```

❶ Output binding created and managed by StreamBridge

❷ The actual name at the broker that the binder binds to (the exchange in RabbitMQ)

All that's left now is calling the method whenever a submitted order is accepted. That's a critical point and one of the aspects characterizing the *saga pattern*, a popular alternative to distributed transactions in microservice architectures. To ensure consistency in your system, persisting an order in the database and sending a message about it must be done atomically. Either both operations succeed, or they both must fail. A simple yet effective way to ensure atomicity is by wrapping the two operations in a local transaction. To do that, we can rely on the built-in Spring transaction management functionality.

**NOTE** The saga pattern is described extensively in chapter 4 of Chris Richardson's book, *Microservices Patterns* (Manning, 2018; https://livebook.manning.com/book/microservices-patterns/chapter-4). I recommend you check it out if you're interested in designing business transactions that span multiple applications.

In the `OrderService` class, modify the `submitOrder()` method to call the `publishOrderAcceptedEvent` method, and annotate it